CS 6515: Graduate Algorithms

Lecture 16: An Introduction to Randomized Algorithms

Instructors: Sahil Singla and Gerandy Brito

Fall 2026

Today's question. What can an algorithm gain by making random choices? We will distinguish two kinds of guarantees, review the probability tools we need, and analyze Karger's remarkably simple randomized minimum-cut algorithm.

1 Why randomize an algorithm?

A *randomized algorithm* makes choices using random bits during its execution. For a fixed input, different executions can follow different paths, take different amounts of time, or even produce different outputs.

What randomization can buy us. Simpler algorithms, good expected running times, and useful success guarantees even when a deterministic choice seems difficult. The guarantees are always taken over the algorithm's random choices, *not* over a presumed random input distribution.

2 Two paradigms: Monte Carlo and Las Vegas

Definition (Las Vegas algorithm). A randomized algorithm that always returns a correct answer (when it terminates), but whose running time is a random variable. We usually analyze its *expected running time*.

Example. Randomized Quicksort: every pivot sequence produces the correctly sorted array; only the running time changes.

Definition (Monte Carlo algorithm). A randomized algorithm with a prescribed running-time bound whose output may be incorrect with some bounded probability. We analyze its *success probability* or *error probability*.

Example. Karger's contraction algorithm always produces a cut, but it need not be a *minimum* cut. Its running time can be bounded, and we will lower-bound the chance that one run returns a minimum cut.

A subtlety. These are useful paradigms, not a claim that every randomized algorithm fits neatly into exactly one box. A Monte Carlo algorithm may have *one-sided error*: for instance, a reported cut has an easily checked capacity, but we may not know whether it is globally minimum.

3 Very Fast Intro to Probability

3.1 Finite probability spaces and random variables

A finite sample space Ω lists possible outcomes. Each $\omega \in \Omega$ has probability $p_\omega \geq 0$, and $\sum_{\omega \in \Omega} p_\omega = 1$. An event $A \subseteq \Omega$ has probability

$$\mathbb{P}(A) = \sum_{\omega \in A} p_\omega.$$

For a fair coin, $\Omega = \{H, T\}$ and both outcomes have probability $1/2$. For a fair die, $\Omega = \{1, \ldots, 6\}$ and each outcome has probability $1/6$.

Definition (Random Variable). A *random variable* $X : \Omega \to \mathbb{R}$ assigns a number to each outcome.

3.2 Three basic random variables

- *Bernoulli R.V* A random variable with exactly two outcome is called a Bernoulli Random Variable. It fits the following situation: imagine running an experiment and declaring success if a desired property occurs (and declare the experiment unsuccessful otherwise).

  We denote it $X \sim Ber(p)$ where p is the probability of success:

  $$\mathbb{P}(X = 1) = p = 1 - \mathbb{P}(X = 0).$$

- *Binomial* Now consider running the experiment independently n times and tracking the number of times it is successful. Alternatively, we can add n Bernoulli random variables:

  $$X = x_1 + X_2 + \ldots + X_n$$

  X has Binomial Distribution, denoted $X \sim Bin(n, p)$ and we have

  $$\mathbb{P}(X = k) = \binom{n}{k} p^k (1-p)^{n-k} \text{ for } 0 \leq k \leq n.$$

- *Geometric* Finally, run the experiment until you observe success. Now our random variable has an infinite countable range. This is the Geometric Distribution, denoted $X \sim Geo(p)$ and

  $$\mathbb{P}(X = k) = (1-p)^{k-1} p \text{ for } k \geq 1.$$

3.3 Expected Value

Definition (Expected Value of a discrete Random Variable).
Let X be a discrete random variable. We define its expectation as

$$\mathbb{E}(X) = \sum x \mathbb{P}(X = x)$$

Theorem (Linearity of expectation). For random variables with finite expectations,

$$\boxed{\mathbb{E}[X_1 + \cdots + X_n] = \mathbb{E}[X_1] + \cdots + \mathbb{E}[X_n].}$$

Important: independence is not required.

Quick example. Toss a fair coin n times and let X count heads. Without enumerating 2^n outcomes, write $X = \sum_i X_i$ and obtain $[X] = n/2$. Generalize this to get the expectation of a Binomial R.V.

3.4 Markov's inequality

Theorem (Markov's inequality). If $X \geq 0$ and $a > 0$, then

$$\boxed{\mathbb{P}(X \geq a) \leq \frac{\mathbb{E}[X]}{a}.}$$

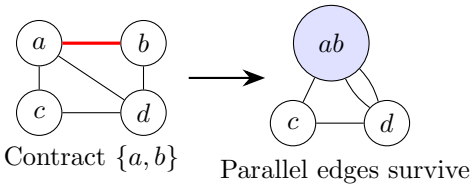
Interpretation. If an algorithm's nonnegative running time has expectation T, the probability that it takes at least $10T$ time is at most $1/10$. Markov is general, but often not very sharp.

Check for understanding. Suppose $\mathbb{E}[X] = 6$ and $X \geq 0$. What does Markov say about $\mathbb{P}(X \geq 24)$? Can it tell us $\mathbb{P}(X \geq 4)$ is small?

4 Karger's randomized minimum-cut algorithm (also named by someone "The Watermelon algorithm")

Let $G = (V, E)$ be a connected, undirected *multigraph*, with $n = |V| \geq 2$. Parallel edges are allowed. For a nonempty proper subset $S \subset V$, the cut $\delta(S)$ consists of edges with one endpoint in S and the other in $V \setminus S$. Its size is $|\delta(S)|$. We seek a global minimum cut of size λ.

Contraction. To contract an edge $\{u, v\}$, merge its endpoints into one supervertex. Delete self-loops, but *keep parallel edges*. Every supervertex represents a set of original vertices.



Karger-Min-Cut(G)

1. While the current multigraph has more than two supervertices:
   (a) Choose one of its current edges *uniformly at random* (counting parallel edges separately).
   (b) Contract that edge and delete any resulting self-loops.
2. Return the partition of the original vertices represented by the two remaining supervertices.

Why this always returns a cut. At the end there are exactly two supervertices, hence a partition of V. But its capacity may exceed λ: the algorithm is Monte Carlo.

4.1 A single run succeeds with probability at least $2/[n(n-1)]$

We notice that if we contract an edge not in a min cut, then such cut is minimum in the contracted graph. We will prove the following

Theorem (Probability of Success). For a fixed minimum cut Δ, one run of Karger's algorithm returns Δ with probability at least

$$\boxed{\mathbb{P}(\text{return } \Delta) \geq \frac{2}{n(n-1)}.}$$

Proof. Fix the edges Δ of some minimum cut. If the algorithm *never contracts an edge of* Δ, the final two supervertices correspond to its two sides, so it returns the min cut.

Let $e_1, e_2, \ldots, e_{n-2}$ the (random!) edges contracted by our algorithm. We have

$$\mathbb{P}(e_1 \notin \Delta) = \frac{|E| - |\Delta|}{|E|} = 1 - \frac{|\Delta|}{|E|}$$

Condition on the event that $\{e_1 \notin \Delta\}$, we can repeat the same calculation, with one less edge:

$$\mathbb{P}(e_2 \notin \Delta | e_1 \notin \Delta) = \frac{|E| - 1 - |\Delta|}{|E| - 1} = 1 - \frac{|\Delta|}{|E| - 1}$$

and in general,

$$\mathbb{P}(e_k \notin \Delta | e_1, \ldots, e_{k-1} \notin \Delta) = \frac{|E| - k + 1 - |\Delta|}{|E| - k + 1} = 1 - \frac{|\Delta|}{|E| - k + 1}$$

We use the following lemma:

Degree bound Let Δ be a min cut in a graph G. Then

$$deg(v) \geq |\Delta| \text{ for all} v \in V$$

Proof The partition $(\{v\}, V \setminus \{v\})$ defines a cut of size $deg(v)$, which is at least the size of the min cut.□

Now we use the equality $2|E| = \sum deg(v)$ in each graph G_e to get that

$$2|E| = \sum deg(v) \geq |\Delta||V| \textbf{ or } \frac{|\Delta|}{|E| - k + 1} \leq \frac{2}{|V| - k + 1}.$$

Multiply over $k = 1, \ldots, n-2$:

$$\begin{aligned}\mathbb{P}(\text{never contract } \Delta) &\geq \prod_{k=1}^{n-2} \left(1 - \frac{2}{|V| - k + 1}\right) = \prod_{r=3}^{n} \frac{r-2}{r} \\ &= \frac{1 \cdot 2 \cdots (n-2)}{3 \cdot 4 \cdots n} = \boxed{\frac{2}{n(n-1)}}.\end{aligned}$$

Therefore the algorithm returns a minimum cut with at least this probability. □

5 What to remember

1. Randomization is over the algorithm's own choices, even on a fixed worst-case input.
2. Las Vegas: always correct, random running time. Monte Carlo: bounded running time, possible error.
3. Linearity of expectation simplifies counting; Markov converts expectations into probability bounds.
4. Karger repeatedly contracts uniformly random edges; parallel edges must be retained.
5. One run finds a fixed minimum cut with probability at least $2/[n(n-1)]$.